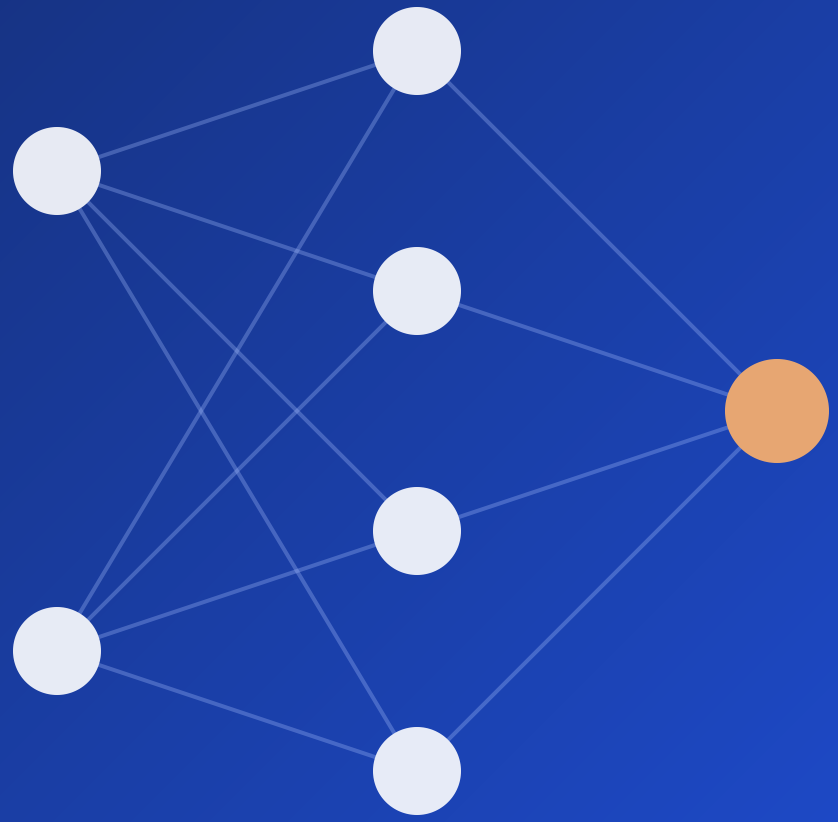
AIML-501 · PORTFOLIO ARTIFACT

Inside a Neural Network

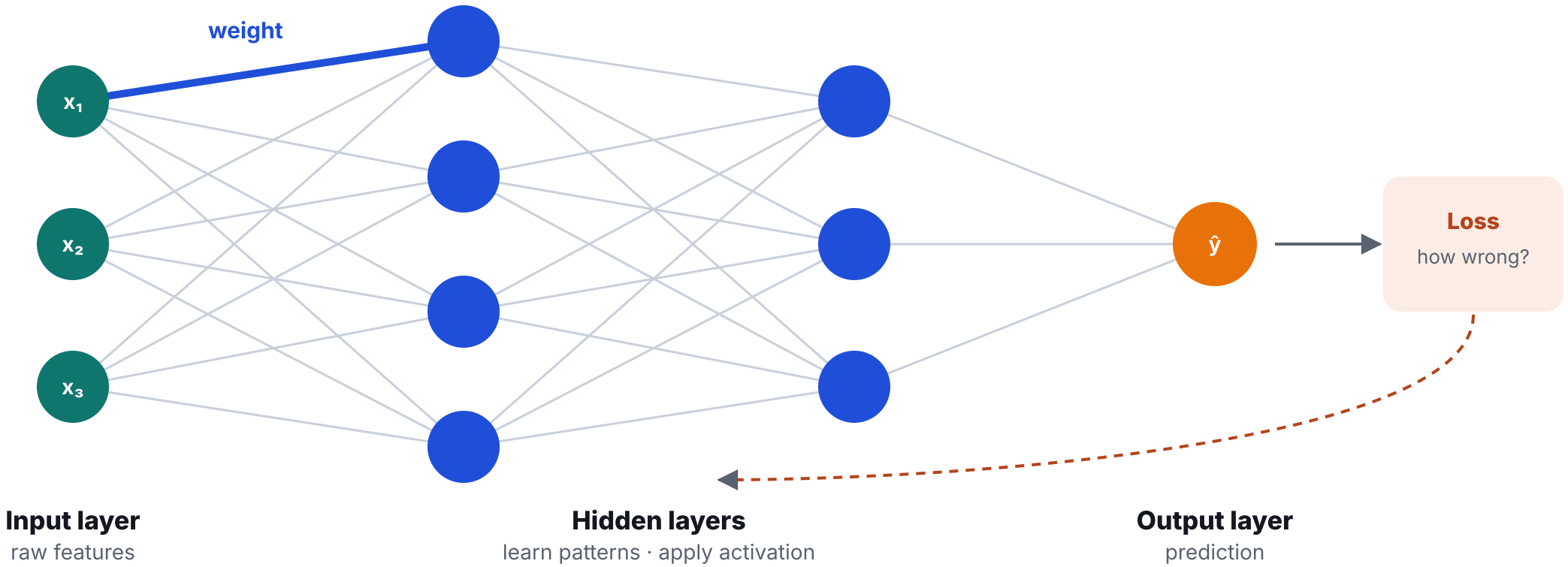
The six building blocks, how data flows through them, and what I learned training my own models in the TensorFlow Playground.

Prashant · Indiana Wesleyan University



THE BIG PICTURE

How data flows through a neural network



Forward pass (left to right): each layer transforms the data and passes it on. **Backward pass** (dashed): the optimizer uses the loss to adjust every weight a little, then repeats.

COMPONENTS 1 & 2

Layers and neurons

Layers

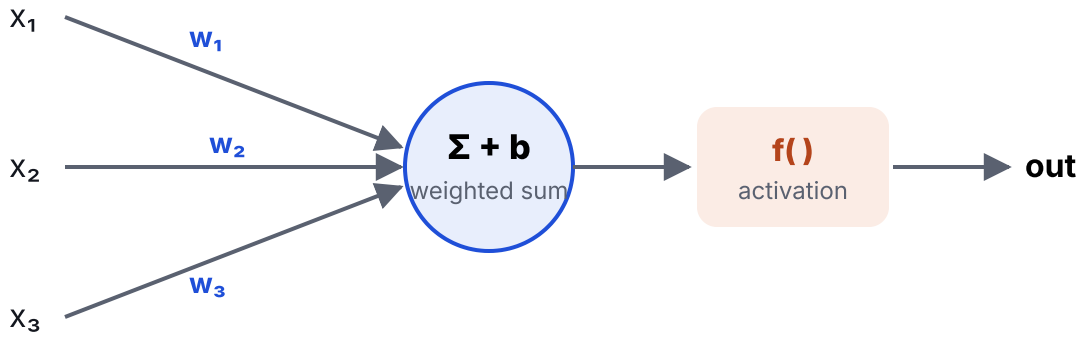
A **layer** is a group of neurons that work at the same step.

- **Input layer** holds the raw features, like pixel values or x and y coordinates.
- **Hidden layers** learn more abstract patterns. Early layers find simple shapes, later ones combine them.
- **Output layer** gives the final answer, like a class or a number.

More layers = a "deeper" network that can learn more complex shapes.

Neurons

A **neuron** is a tiny calculator. It takes inputs, multiplies each by a weight, adds them up with a bias, then applies an activation function.



output = f($w_1x_1 + w_2x_2 + w_3x_3 + b$)

COMPONENT 3

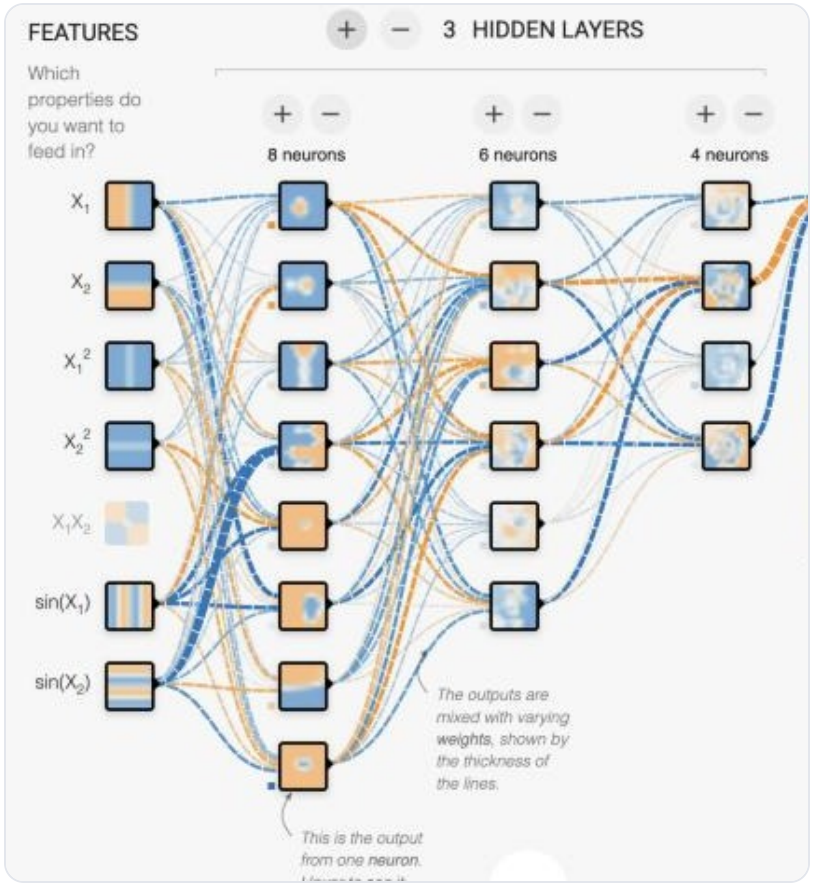
Weights (and biases)

A **weight** is a number on each connection that says how much one neuron's output matters to the next. A **bias** is an extra number each neuron adds, so it can shift its output up or down.

- Large positive weight → strong "yes" signal
- Negative weight → pushes the other way
- Near zero → that input is basically ignored

Weights start out random. **Training is just the process of finding good weights.** Everything the network "knows" is stored in them.

In the Playground, line thickness shows the size of each weight and color shows the sign (blue positive, orange negative).

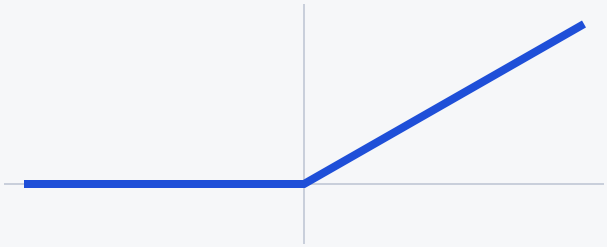


My 3-hidden-layer network (8-6-4 neurons) in the TensorFlow Playground. Thicker lines are bigger weights.

COMPONENT 4

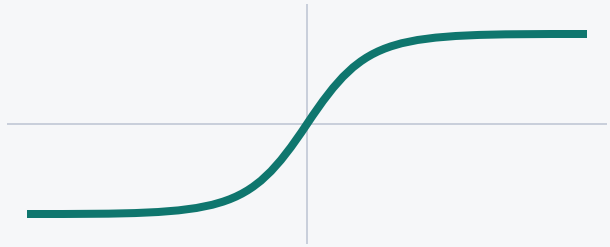
Activation functions

After a neuron adds up its inputs, the **activation function** decides what to pass on. It adds **non-linearity**, which lets the network learn curves instead of only straight lines. Without it, stacking layers would still just make one straight line.



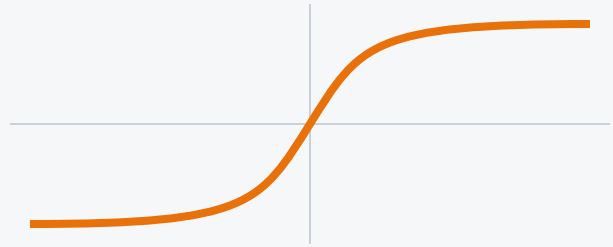
ReLU

Outputs 0 for negatives, the value itself for positives. Fast and the default for most hidden layers.



Sigmoid

Squashes any number to between 0 and 1. Useful for a probability in yes/no outputs.



Tanh

Like sigmoid but from −1 to 1, centered on zero. Worked well for my spiral network.

COMPONENT 5

Loss functions

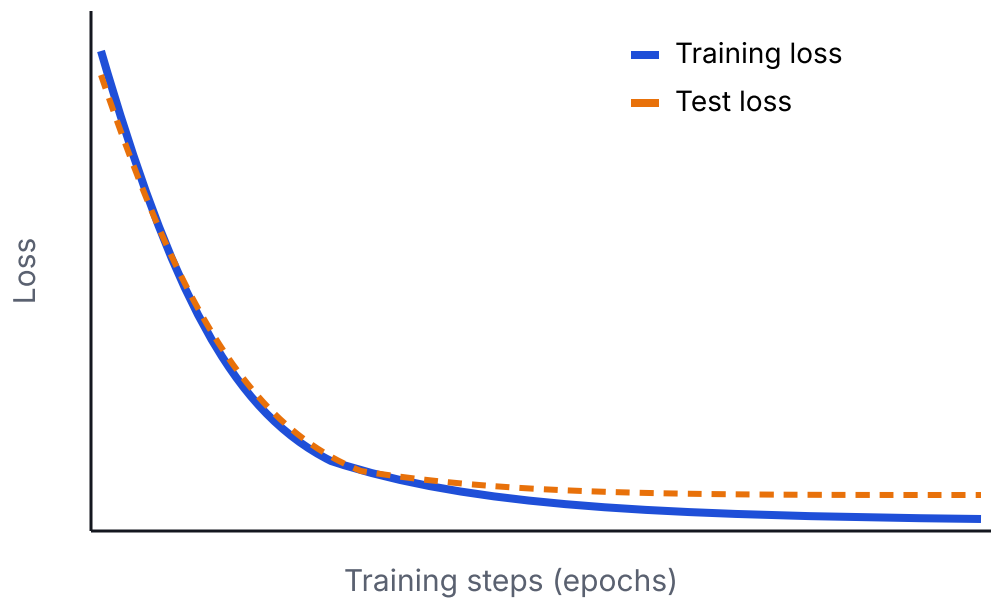
The **loss function** measures how wrong the network's predictions are, as a single number. Lower is better, and training tries to push it toward zero.

Mean Squared Error (MSE)

Average of (prediction − actual)². Used for predicting numbers, like a price. Big mistakes are punished a lot.

Cross-Entropy

Used for classification. Punishes the model heavily when it is confident and wrong.



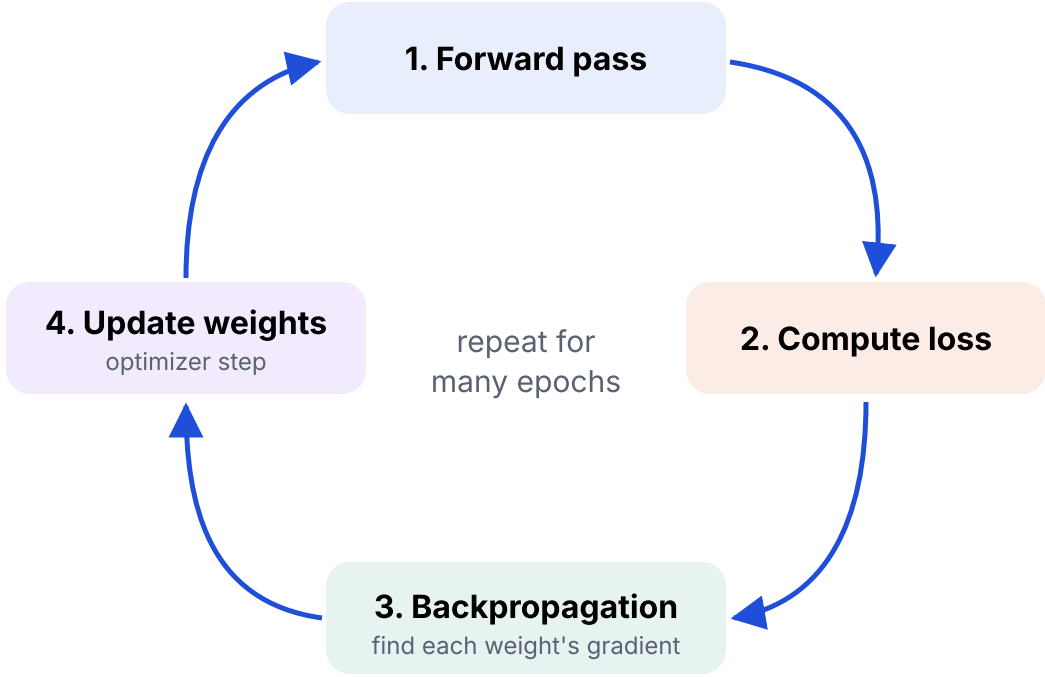
A healthy loss curve: both drop quickly and level off. If test loss starts rising while training loss keeps falling, the model is overfitting.

COMPONENT 6

Optimization algorithms

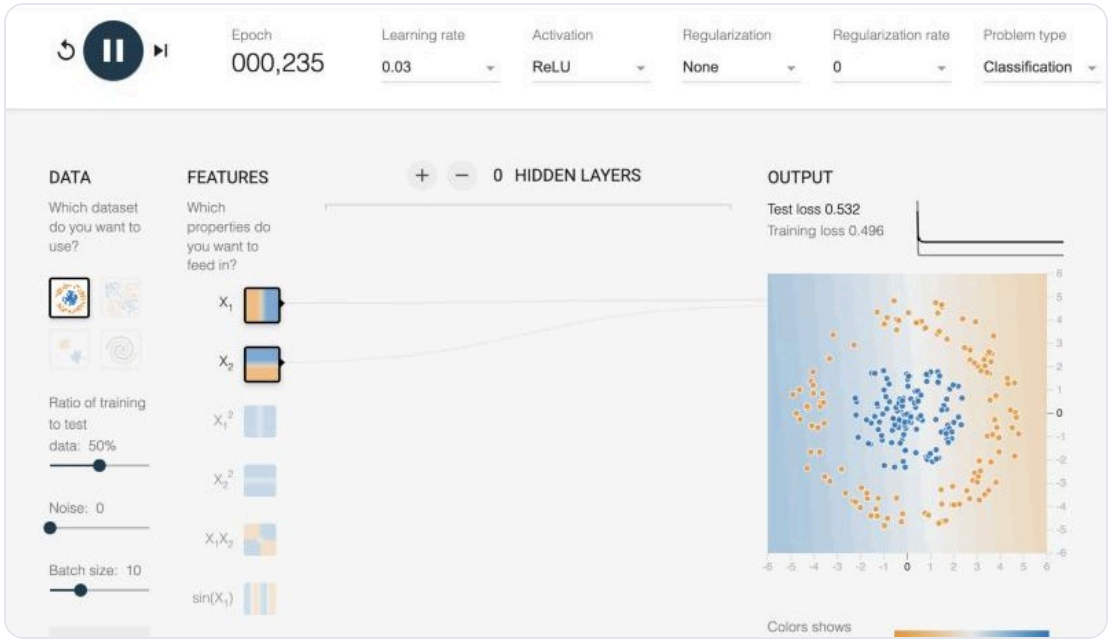
The **optimizer** decides how to change the weights to lower the loss. The core idea is **gradient descent**: like walking downhill in fog, take a small step in whichever direction slopes down most steeply.

- **Learning rate** = step size. Too big and it overshoots, too small and it takes forever.
- **SGD** updates using small batches of data at a time.
- **Adam** adapts the step size for each weight automatically. A popular default.



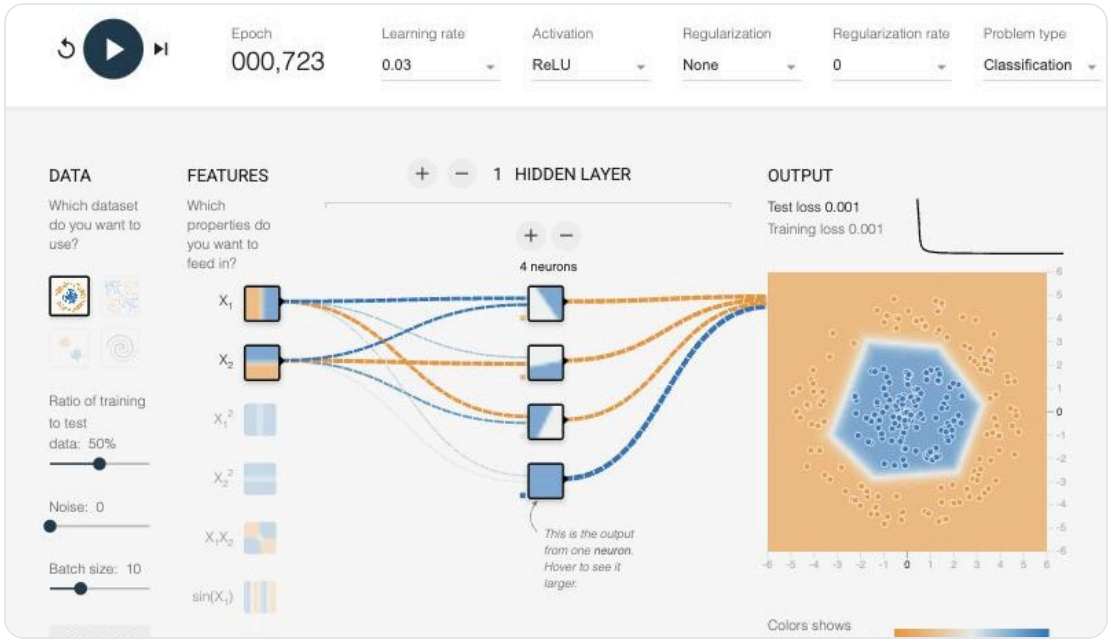
TENSORFLOW PLAYGROUND · EXPERIMENT 1

Why hidden layers matter



0 hidden layers → test loss 0.53

With no hidden layer the model can only draw a straight line, so it can't separate a circle from the ring around it.

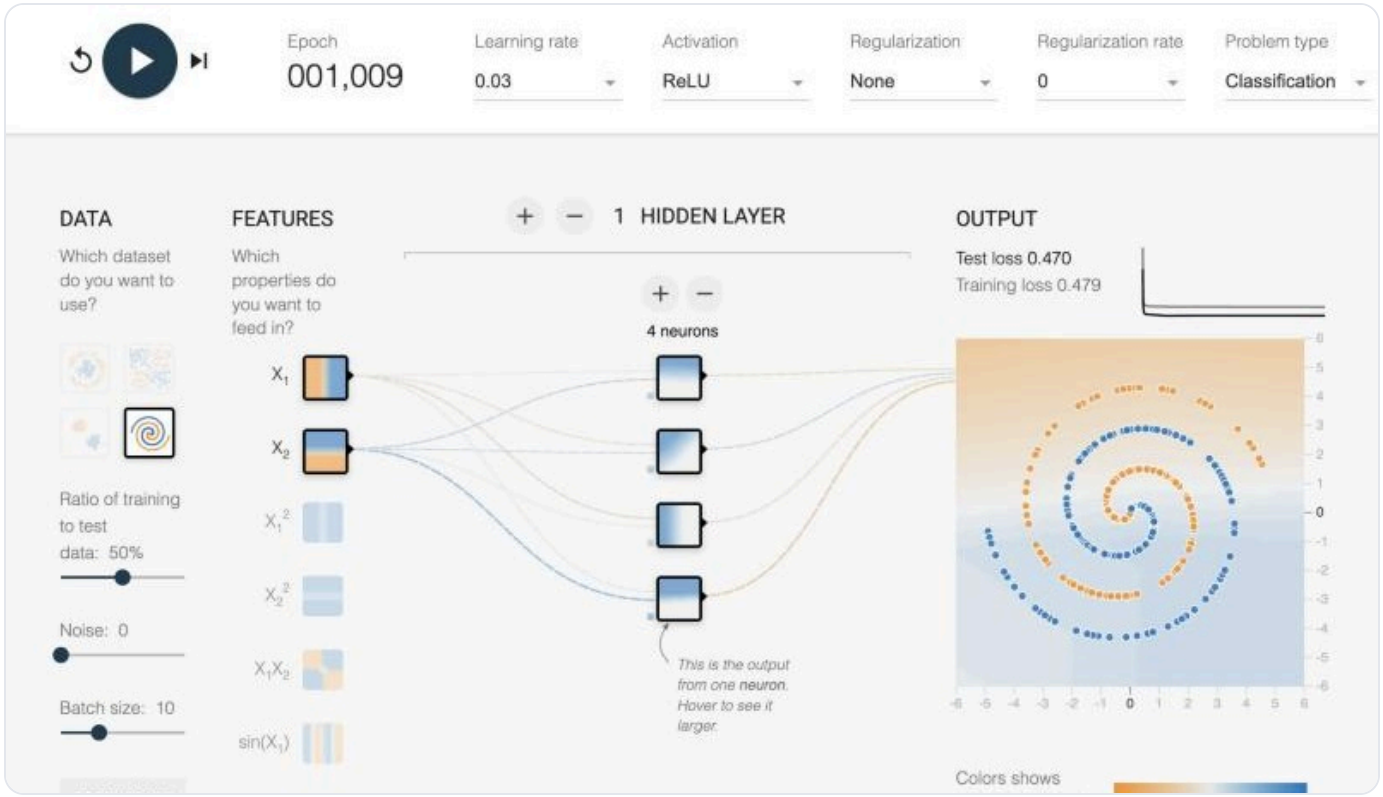


1 hidden layer, 4 neurons (ReLU) → test loss 0.001

Adding just 4 neurons lets the network combine straight lines into a closed shape around the blue points, in about 700 epochs.

TENSORFLOW PLAYGROUND · EXPERIMENT 2

Harder data needs more capacity



Spiral, same 1×4 network → test loss 0.47 after 1,000 epochs

The network that solved the circle couldn't learn the spiral at all. More training time didn't help, because the model was simply too small.

What I changed next

- 3 hidden layers (8, 6, 4 neurons)
- Tanh activation
- Extra input features: x^2, y^2, sin(x), sin(y)

This bigger setup (shown on slide 4) has far more weights to tune. Each epoch takes longer, and the network is much harder to interpret, but it gives the model the capacity to bend around a spiral.

SUMMARY

Why visualizing neural networks matters

It turns a black box into something you can see

Watching weights change thickness and the output map reshape made it clear that training is just adjusting numbers to reduce loss.

Model size should match the problem

0 hidden layers failed on the circle and 1 small layer failed on the spiral. Too little capacity can't be fixed by training longer.

Features and activations matter as much as depth

Good input features like x^2 or sin(x) and the right activation can make a hard problem much easier.

Watch the loss, not just accuracy

Comparing training and test loss is the quickest way to spot underfitting or overfitting.

Key takeaway: every neural network, from the Playground to ChatGPT, is built from the same six parts. Layers of neurons, connected by weights, shaped by activations, scored by a loss, and improved by an optimizer.

Tools: TensorFlow Playground (playground.tensorflow.org)